AULA 04 · 06/11/2026

Os dados aparecem na página

Evento submit, FormData e resumo local com JavaScript

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Escutar submit no formulário e impedir a navegação na simulação.

• Coletar campos por name, preservando múltiplos checkboxes.

• Montar um resumo com elementos do DOM e textContent.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

O evento pertence ao formulário

Usar submit permite tratar o botão e a submissão pelo teclado. A validação HTML interativa acontece antes do submit: campos inválidos podem impedir que esse evento aconteça.

preventDefault interrompe a ação padrão nesta demonstração. Ele não valida, não envia para um servidor e não grava os dados. A base já oferece um bloqueio de navegação; substitua a mensagem pelo processamento do resumo.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  mostrarResumo(dados);
});
```

Coleta por name

new FormData(form) monta os pares de dados dos controles participantes. get recupera um valor; getAll recupera todos os valores com o mesmo name. Checkboxes desmarcados não entram na coleta.

Um campo com id e sem name continua aparecendo na tela, mas não entra como par nome/valor na coleta. Os sintomas selecionados precisam aparecer juntos no resumo.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Resumo como conteúdo, sem interpretar HTML

Crie dt e dd para cada item da lista de descrição. textContent apresenta o valor como texto. Não use innerHTML para inserir o que a pessoa digitou.

Limpe o resumo anterior antes de montar o novo. Depois torne a seção visível e leve o foco ao título com tabindex=-1. O resumo corresponde à última simulação, não a um registro persistente.

```
function adicionarLinha(titulo, valor) {
  const dt = document.createElement('dt');
  const dd = document.createElement('dd');
  dt.textContent = titulo;
  dd.textContent = String(valor);
  resumo.append(dt, dd);
}
```

Estado do formulário depois da tentativa

Uma falha de validação deve manter os valores para correção. Um novo envio deve substituir o resumo anterior, sem duplicar linhas.

O botão Novo preenchimento é type=button. Ele pode chamar reset, ocultar o resumo e devolver o foco ao primeiro campo. Avise que o projeto perde os dados quando a página é recarregada.

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Preparar o resultado

Onde inserir: index.html: depois de form, crie section#resultado hidden com h2#titulo-resumo tabindex="-1" e dl#dados-resumo. Dentro de div.acoes do form, acrescente button#novo type="button".

Entenda: Crie a área para o resumo e mantenha Novo preenchimento fora da área escondida.

```
<!-- Depois do fechamento de form: -->
<section class="painel" id="resultado" hidden
  aria-labelledby="titulo-resumo">
  <h2 id="titulo-resumo" tabindex="-1">Resumo da última simulação</h2>
  <p>Nenhuma solicitação foi enviada ou gravada em um servidor.</p>
  <dl id="dados-resumo"></dl>
</section>

<!-- Dentro de div.acoes do form, depois do botão submit: -->
<button type="button" id="novo">Novo preenchimento</button>
```

Resultado esperado: A área do resumo começa oculta; Novo preenchimento permanece acessível.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Coletar no submit

Onde inserir: script.js: substitua o manipulador de submit fornecido; mantenha um único listener.

Entenda: Cancele a navegação; crie new FormData(form); use get e getAll.

```
const resultado = document.querySelector('#resultado');
const resumo = document.querySelector('#dados-resumo');
// No único listener de submit, após preventDefault():
const dados = new FormData(form);
mostrarResumo(dados);
```

Resultado esperado: Ao marcar dois sintomas, ambos permanecem na lista coletada.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Mostrar e reiniciar

Onde inserir: script.js: implemente mostrarResumo, adicionarLinha e o clique de #novo conforme os exemplos do conteúdo.

Entenda: Use createElement, textContent e replaceChildren; revele o resultado e leve o foco ao título. Mude o texto do botão submit para Gerar resumo local.

Resultado esperado: Um envio válido mostra resumo local. Novo preenchimento limpa e foca o solicitante.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Substituir o tratamento de submit pela coleta com FormData e pela chamada que monta o resumo.

3. Mostrar solicitante, equipamento, contato, categoria, prioridade, descrição e todos os sintomas selecionados.

4. Acrescentar a limpeza do resumo em uma nova simulação e o botão de novo preenchimento.

Casos de teste

Caso	Resultado esperado
Campo obrigatório vazio	O resumo não aparece.
Preenchimento válido	Resumo local com os valores informados.
Dois sintomas marcados	Ambos aparecem no resumo.
Nenhum sintoma marcado	Mensagem de nenhuma opção selecionada.
Texto com exemplo na descrição	As marcas aparecem como texto, sem formatação HTML.
Duas simulações válidas	O resumo mais recente substitui o anterior.
Novo preenchimento	Campos reiniciados e foco no primeiro campo.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Como o name chega ao FormData? Por que usar textContent no resumo?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: evento submit (em inglês)

MDN: validação de formulários

MDN: uso de FormData (em inglês)

MDN: textContent (em inglês)

MDN: elemento button (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)